TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 3: Graphs, Graph Traversal, and Union-Find

Foreign Trade University

Contents

1 Introduction

This week we move from the single case study of Week 1 (stable matching) to the most pervasive modeling tool in algorithm design: **graphs**. An enormous range of problems – road networks, social networks, dependency structures, web link graphs, scheduling constraints – can be modeled as graphs, and many fundamental algorithmic questions ("is there a path?", "what is the shortest path?", "is there a cycle?", "can these tasks be ordered consistently?") reduce to graph traversal.

We follow Kleinberg & Tardos, Chapter 3 (Graphs) and Appendix B (Union-Find / Disjoint Sets). The plan for this week:

1. **Graph representations** – adjacency lists vs. adjacency matrices, and the space/time trade-offs between them.
2. **Breadth-First Search (BFS)** – shortest paths in unweighted graphs, layers, and connected components.
3. **Depth-First Search (DFS)** – recursive and iterative formulations, connected components, and cycle detection.
4. **Bipartiteness** – 2-coloring via BFS, and the odd-cycle characterization.
5. **Directed graphs and DAGs** – cycle detection with 3-color DFS, and **topological sort** via DFS and via Kahn's algorithm.
6. **Union-Find (Disjoint Sets)** – quick-find, quick-union, union by rank, path compression, and the resulting near-constant amortized cost per operation.

2 Graphs: Definitions and Representations

Definition 2.1 (Graph)**.** *A graph $G = (V, E)$ consists of a set of **vertices** (or **nodes**) V and a set of **edges** E. In an **undirected** graph, each edge $\{u, v\} \in E$ is an unordered pair, representing a symmetric relationship. In a **directed** graph (or **digraph**), each edge $(u, v) \in E$ is an ordered pair (an* arc*) from u to v.*

We write $n = |V|$ for the number of vertices and $m = |E|$ for the number of edges. Throughout this course, vertices are typically labeled $0, 1, \ldots, n-1$.

Definition 2.2 (Degree, self-loops, multi-edges)**.** *The **degree** of a vertex v in an undirected graph is the number of edges incident to v; a **self-loop** (v, v) contributes 2 to $\deg(v)$ by convention. A **simple graph** has no self-loops and no **multi-edges** (parallel edges between the same pair of vertices).*

Lemma 2.3 (Handshake Lemma)**.** *For any undirected graph, $\sum_{v \in V} \deg(v) = 2|E|$.*

Proof. Every edge $\{u, v\}$ (with $u \neq v$) contributes exactly 1 to $\deg(u)$ and 1 to $\deg(v)$, for a total contribution of 2; a self-loop (v, v) contributes 2 directly to $\deg(v)$ by convention. Either way, each edge contributes exactly 2 to the sum of degrees. □

Problem 2 asks you to implement `degree_sequence`, `has_self_loop`, `has_multi_edge`, and a `handshake_check` that verifies this lemma empirically.

2.1 Adjacency list vs. adjacency matrix

Definition 2.4 (Adjacency list)**.** *An **adjacency list** represents G as an array* adj$[0..n-1]$ *where* adj$[v]$ *is the list of vertices adjacent to v (for directed graphs,* adj$[v]$ *lists the* out-neighbors *of v).*

Definition 2.5 (Adjacency matrix)**.** *An **adjacency matrix** represents G as an $n \times n$ matrix A where $A[u][v]$ is the number of edges from u to v (for a simple graph, $A[u][v] \in \{0, 1\}$; for an undirected simple graph, A is symmetric).*

Operation	Adjacency list	Adjacency matrix
Space	$\Theta(n+m)$	$\Theta(n^2)$
Is $(u, v) \in E$?	$O(\deg(u))$	$O(1)$
Iterate over neighbors of v	$\Theta(\deg(v))$	$\Theta(n)$
Add an edge	$O(1)$	$O(1)$

Table 1: Adjacency list vs. adjacency matrix.

Sparse vs. dense graphs. A graph is **sparse** if $m = O(n)$ (or more generally $m \ll n^2$), and **dense** if $m = \Theta(n^2)$. For sparse graphs (the common case for real-world networks: road networks, social networks, the web graph), the adjacency list's $\Theta(n+m)$ space is dramatically smaller than the matrix's $\Theta(n^2)$, and traversal algorithms (BFS/DFS) that visit each edge once run in $O(n+m)$ time on an adjacency list – this is the representation we use by default. Problem 1 asks you to build both representations from an edge list *from scratch*; Problem 3 asks you to convert between them and compute `matrix_density` to make this trade-off concrete.

3 Breadth-First Search (BFS)

Breadth-First Search explores a graph "layer by layer" outward from a source vertex s: first all vertices at distance 1 from s, then all vertices at distance 2, and so on. It uses a queue (FIFO), which is exactly why we studied queues in Week 1.

Algorithm 1 BFS(adj, s)

```
1: dist[s] ← 0; dist[v] ← ∞ (or −1) for all v ≠ s
2: Q ← empty queue; enqueue s
3: while Q is not empty do
4:     u ← dequeue(Q)
5:     for each v ∈ adj[u] do
6:         if dist[v] is still ∞ (unvisited) then
7:             dist[v] ← dist[u] + 1
8:             enqueue(v)
9:         end if
10:    end for
11: end while
12: return dist
```

Theorem 3.1 (BFS computes shortest-path distances)**.** *In an unweighted graph (directed or undirected), the value dist$[v]$ computed by BFS from s equals the length (number of edges) of a shortest*

path from s to v, for every v reachable from s; and v is reachable from s iff $dist[v]$ is finite (i.e. v was ever enqueued).

Proof sketch. By induction on the distance k. The set of vertices with $dist[v] = 0$ is $\{s\}$, correct by definition. Assume that for all $j < k$, $dist[v] = j$ exactly for vertices at true distance j from s, and that BFS has enqueued (in some order) exactly the vertices at distances $0, \ldots, k-1$ before dequeuing any vertex at distance $\geq k$ – this follows because BFS processes a FIFO queue, so all distance-$(j-1)$ vertices are dequeued (and their neighbors examined) before any distance-j vertex is enqueued.

Now consider a vertex v at true distance k from s. By definition of distance, v has a neighbor u at true distance $k-1$ (the second-to-last vertex on a shortest s-v path), and v has no neighbor at distance $< k-1$ (otherwise v would have distance $< k$). By the inductive hypothesis, u is dequeued with $dist[u] = k-1$ before any distance-k vertex is processed, and at that point v is still unvisited (since none of v's neighbors at distance $\leq k-1$ other than possibly u have been processed yet to set $dist[v]$ – and if one had, it would imply $dist[v] \leq k-1 < k$, a contradiction). So v is discovered while processing u, with $dist[v] \leftarrow dist[u] + 1 = k$, which is correct. Vertices not reachable from s are never enqueued, so $dist[v]$ remains ∞ (or -1). □

Running time. BFS runs in $O(n+m)$ time on an adjacency list: each vertex is enqueued at most once ($O(n)$ total dequeues), and the total work in the inner `for` loop over all dequeues is $\sum_u |adj[u]| = O(m)$ (each edge is examined at most twice for undirected graphs, once for directed graphs).

BFS layers. The sequence $L_0 = \{s\}, L_1, L_2, \ldots$ where $L_k = \{v : dist[v] = k\}$ is called the **BFS layering**. Problem 4 asks you to compute both `dist` and the explicit layers `bfs_layers`.

Shortest path reconstruction. To recover an actual shortest path (not just its length), record a **parent pointer** $parent[v] = u$ whenever v is first discovered via u. The path $s \to \cdots \to t$ is obtained by following parent pointers from t back to s and reversing. Problem 5 implements `bfs_shortest_path`.

3.1 Connected components via BFS

Definition 3.2 (Connected component)**.** *In an undirected graph, two vertices u, v are in the same **connected component** if there is a path between them. The connected components partition V into disjoint subsets.*

Algorithm 2 Connected components via BFS

```
mark all vertices unvisited
for v = 0, 1, ..., n − 1 do
    if v unvisited then
        run BFS from v, marking everything it reaches as visited and as part of a new component
    end if
end for
```

Running BFS from every unvisited vertex costs $O(n+m)$ total (each vertex/edge is touched a constant number of times across all BFS calls combined). Problem 6 implements `connected_components_bfs` and `num_components`.

3.2 Graph diameter

Definition 3.3 (Eccentricity and diameter)**.** *The* ***eccentricity*** *of v is $\max_{u \in V} dist(v, u)$. The* ***diameter*** *of a connected graph is $\max_{v \in V} \mathrm{ecc}(v) = \max_{u,v} dist(u, v)$, the longest shortest-path distance between any pair of vertices.*

Computing the diameter exactly by running BFS from *every* vertex costs $O(n(n+m))$ – this is the "all-pairs shortest paths via repeated BFS" approach, only practical for moderately sized graphs. Problem 7 implements `eccentricity` and `diameter` this way.

4 Depth-First Search (DFS)

Depth-First Search explores as far as possible along each branch before backtracking. It is naturally recursive (using the call stack) but can also be implemented iteratively with an explicit stack.

Algorithm 3 DFS-Visit(adj, u) – recursive

```
1: mark u visited; record u in the preorder
2: for each v ∈ adj[u] do
3:     if v unvisited then
4:         DFS-Visit(adj, v)
5:     end if
6: end for
```

Calling `DFS-Visit`(adj, s) visits exactly the vertices reachable from s, each exactly once, in $O(|\text{reachable}| + |\text{their edges}|)$ time – so DFS from a single source costs $O(n+m)$ in the worst case, same asymptotic bound as BFS. The *order* of visitation differs (DFS goes deep first; BFS goes wide first), but the *set* of reachable vertices is identical, and both run in $O(n+m)$.

Problem 8 implements the recursive version, `dfs_recursive`, returning the preorder (discovery order).

4.1 Discovery and finish times

For many applications (especially cycle detection and topological sort) it is useful to record, for each vertex u, a **discovery time** $d[u]$ (when DFS first visits u) and a **finish time** $f[u]$ (when DFS finishes processing all of u's descendants and returns from the recursive call). These are obtained with a single global counter incremented on every discovery and every finish event. Problem 8 also implements `dfs_discovery_finish_times`.

Proposition 4.1 (Parenthesis structure)**.** *For any two vertices u, v visited during the same DFS, the intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ either are disjoint, or one is nested entirely inside the other – they never "partially overlap". This is the basis for classifying edges as tree, back, forward, or cross edges.*

4.2 Iterative DFS with an explicit stack

Recursive DFS uses $O(h)$ call-stack frames, where h is the depth of the recursion (which can be $\Theta(n)$ for a path graph, risking stack overflow in languages/runtimes with limited recursion depth). An **iterative** version uses an explicit stack (a list with `append`/`pop`):

Algorithm 4 Iterative DFS(adj, s)

```
1: stack ← [s]; mark nothing visited yet
2: while stack not empty do
3:     u ← stack.pop()
4:     if u not yet visited then
5:         mark u visited; record u in the preorder
6:         push adj[u] onto the stack in reverse order (so the first neighbor is popped first, matching
   the recursive order)
7:     end if
8: end while
```

Problem 9 implements `dfs_iterative` (matching the discovery order of `dfs_recursive`) and `max_stack_size`, which measures the $O(n)$ worst-case auxiliary space DFS can use (e.g. on a path graph, the stack can grow to size $\Theta(n)$).

4.3 Connected components via DFS

Exactly as with BFS, running DFS from every unvisited vertex (in order $0, 1, \ldots, n-1$) partitions V into connected components in $O(n+m)$ total time. Problem 10 implements `connected_components_dfs`, `same_component`, and `is_connected`, and observes that DFS and BFS produce the *same partition* of vertices into components (only the traversal order differs).

4.4 Cycle detection in undirected graphs

Algorithm 5 Cycle detection (undirected) via DFS

```
1: function DFS-Visit(u, parent)
2:     mark u visited
3:     for each v ∈ adj[u] do
4:         if v unvisited then
5:             DFS-Visit(v, u)
6:         else if v ≠ parent then
7:             return "cycle found"                ▷ back edge to a non-parent visited vertex
8:         end if
9:     end for
10: end function
```

Proposition 4.2. *An undirected graph contains a cycle iff, for some unvisited vertex u during a DFS, u has a neighbor v that is already visited and is* not *u's parent in the DFS tree (a **back edge**).*

The subtlety: in an adjacency list built from undirected edges, each edge $\{u, v\}$ appears as $v \in adj[u]$ *and* $u \in adj[v]$. When DFS is at u and sees $v = parent[u]$, this is just the edge it arrived on, *not* a cycle – it must be excluded explicitly. Problem 11 implements `has_cycle_undirected` and `find_a_cycle` (which reconstructs an explicit cycle).

5 Bipartiteness

Definition 5.1 (Bipartite graph)**.** *An undirected graph $G = (V, E)$ is **bipartite** if V can be partitioned into two sets A, B such that every edge has one endpoint in A and one in B. Equivalently, G admits a proper **2-coloring**: a function* $\text{color} : V \to \{0, 1\}$ *such that* $\text{color}(u) \neq \text{color}(v)$ *for every edge* $\{u, v\}$*.*

5.1 2-coloring via BFS

Algorithm 6 2-Color(*adj*)

```
for each unvisited vertex s do
    color[s] ← 0; enqueue s
    while queue not empty do
        u ← dequeue
        for each v ∈ adj[u] do
            if v unvisited then
                color[v] ← 1 − color[u]; enqueue v
            else if color[v] = color[u] then
                return "not bipartite"
            end if
        end for
    end while
end for
return color
```

Problem 12 implements `two_color_bfs` (returning `None` if not bipartite) and `is_bipartite`.

5.2 The odd-cycle characterization

Theorem 5.2. *An undirected graph G is bipartite if and only if it contains no cycle of odd length.*

Proof sketch. ($\Rightarrow$) If G is bipartite with parts A, B, then any cycle alternates between A and B as it traverses each edge, so it must use an even number of edges to return to its starting part.

($\Leftarrow$) If G has no odd cycle, 2-color it via BFS from each component: assign $\text{color}(s) = 0$ for the BFS root s and $\text{color}(v) = dist(s, v) \bmod 2$ for every other v. If some edge $\{u, v\}$ had $\text{color}(u) = \text{color}(v)$, then $dist(s, u)$ and $dist(s, v)$ have the same parity, and the cycle formed by the BFS-tree path $s \to u$, the edge u-v, and the path $v \to s$ has length $dist(s, u) + dist(s, v) + 1$, which is *odd* (sum of two equal-parity numbers is even, plus 1 is odd) – contradicting the assumption of no odd cycle. So no such edge exists, and the 2-coloring is proper. □

Problem 13 implements `find_odd_cycle`, which produces an explicit odd cycle as a *certificate* of non-bipartiteness whenever BFS 2-coloring fails – precisely by the construction in the proof above (splicing the two root-to-u and root-to-v paths together with the offending edge).

6 Directed Graphs

In a **directed graph** (digraph), $adj[u]$ lists only the **out-neighbors** of u (vertices v such that (u, v) is an arc). BFS and DFS both work unchanged on digraphs – they simply follow arcs in their given direction – but the notions of "cycle" and "connectivity" become directional.

6.1 Cycle detection via 3-color DFS

Definition 6.1 (Vertex colors during DFS)**.** ***White****: not yet visited.* ***Gray****: currently on the recursion stack (an ancestor of the current vertex in the DFS tree).* ***Black****: finished (popped off the stack, all descendants explored).*

Theorem 6.2. *A directed graph has a directed cycle if and only if some DFS encounters an edge (u, v) where v is* ***gray*** *at the time (u, v) is examined (a* ***back edge*** *to an ancestor).*

Proof sketch. If DFS finds edge (u, v) with v gray, then v is an ancestor of u in the current DFS tree, so there is a path $v \to \cdots \to u$ in the tree, and the edge (u, v) closes this into a directed cycle $v \to \cdots \to u \to v$.

Conversely, if G has a directed cycle $v_0 \to v_1 \to \cdots \to v_{k-1} \to v_0$, consider the first vertex v_i on the cycle that DFS discovers. At the moment v_i is discovered (colored gray), all other v_j on the cycle are white (not yet discovered, else they would have been discovered first by following the cycle, or v_i would not be first). DFS from v_i will eventually traverse $v_i \to v_{i+1} \to \cdots \to v_{i-1} \to v_i$ (the cycle, relabeled to start at v_i); when the edge (v_{i-1}, v_i) is examined, v_i is still gray (its DFS call has not yet returned, since it is the ancestor of everything on this path) – so this edge is a back edge to a gray vertex.

Edges to **black** vertices are *forward* or *cross* edges and never indicate a cycle, since a black vertex is fully finished and cannot be an ancestor of the current vertex. □

Problem 14 implements `has_cycle_directed` (3-color DFS) and `find_a_directed_cycle` (reconstructing an explicit directed cycle from the DFS recursion stack when a back edge is found).

6.2 Topological sort

Definition 6.3 (Topological order)**.** *A* ***topological order*** *of a directed graph $G = (V, E)$ is a linear ordering of V such that for every edge $(u, v) \in E$, u appears before v in the order.*

Theorem 6.4. *A directed graph G has a topological order if and only if G is a* ***DAG*** *(Directed Acyclic Graph, i.e. has no directed cycle).*

Proof sketch. If G has a directed cycle $v_0 \to v_1 \to \cdots \to v_{k-1} \to v_0$, no linear order can place every v_i before $v_{i+1 \bmod k}$ for all i simultaneously (this would require v_0 before v_1 before … before v_{k-1} before v_0, an impossible cyclic ordering constraint). Conversely, if G is a DAG, one can always find a **source** – a vertex with in-degree 0 (otherwise, following in-edges backward forever would either cycle, contradicting acyclicity, or the graph would be infinite). Place this source first, remove it, and recurse on the remaining DAG (which is still acyclic and smaller). This greedy "peel off a source" process is exactly Kahn's algorithm below. □

6.2.1 Topological sort via DFS

Proposition 6.5. *If G is a DAG, the order in which vertices* finish *during DFS, reversed, is a valid topological order.*

Proof sketch. Consider any edge (u, v). We must show u finishes *after* v (so that u comes *before* v in the reversed-finish order). When DFS examines edge (u, v): if v is white, DFS recurses into v and v finishes before u does (since u's call cannot finish until the recursive call on v returns). If v is already black (finished), then v finished before u examined the edge, hence before u finishes. v cannot be gray when (u, v) is examined, because that would mean v is an ancestor of u, giving a

Algorithm 7 Topological-Sort-DFS(adj)

result $\leftarrow$ empty list
for each unvisited vertex u **do**
    DFS-Visit(u), where on *finishing* u (after all its out-neighbors are fully processed), **prepend** u to result (or append and reverse at the end)
end for
return result (or `None` if a cycle was detected during DFS)

back edge and hence a cycle (contradicting that G is a DAG, by the previous theorem). So in all cases v finishes before u, i.e. u finishes after v. □

Problem 15 implements `topological_sort_dfs` (returning `None` on a cycle) and `verify_topological_order`.

6.2.2 Topological sort via Kahn's algorithm (BFS-style)

Algorithm 8 Kahn(adj)

compute $indeg[v]$ = number of incoming edges to v, for all v
$Q \leftarrow$ queue of all v with $indeg[v] = 0$
result $\leftarrow$ empty list
while Q not empty **do**
    $u \leftarrow$ dequeue(Q); append u to result
    for each $v \in adj[u]$ **do**
        $indeg[v] \leftarrow indeg[v] - 1$
        if $indeg[v] = 0$ **then**
            enqueue(v)
        end if
    end for
end while
return result if |result| $= n$, else `None` (cycle detected)

Kahn's algorithm is the direct algorithmic translation of the "repeatedly remove a source" argument in the proof of Theorem 6.4: a vertex with in-degree 0 (no remaining incoming edges from un-placed vertices) is exactly a valid "next" vertex. If the queue empties before all n vertices are output, the remaining vertices all have positive in-degree from each other – i.e. they form a cycle. Both DFS-based and Kahn's algorithm run in $O(n + m)$ time. Problem 16 implements `in_degrees` and `topological_sort_kahn` (made deterministic by always breaking ties in increasing order of vertex index, using a `deque`).

6.2.3 Counting and uniqueness of topological orders

A DAG can have many valid topological orders (e.g. two completely independent chains can be interleaved in any order). Problem 17 implements `count_topological_orders` by backtracking over the set of currently-available (in-degree-0-among-remaining) vertices, and `has_unique_topological_order`. A DAG has a *unique* topological order iff every pair of consecutive vertices in that order is connected by a direct edge – equivalently, the DAG contains a Hamiltonian path consistent with E.

7 Union-Find (Disjoint-Set Union)

The **Union-Find** (or **Disjoint-Set Union**, DSU) data structure maintains a partition of a set of elements into disjoint subsets, supporting two operations:

- `find`(x): return a canonical representative of the subset containing x (so that `find`(x) = `find`(y) iff x and y are in the same subset).
- `union`(x, y): merge the subsets containing x and y into one.

This is exactly the abstraction needed for incremental connectivity: "are u and v connected if we only ever *add* edges?" (Kruskal's MST algorithm, to be covered in a later week, is the canonical application.)

7.1 Quick-Find

Quick-Find stores an array $id[0..n-1]$ where $id[i]$ is the component id of element i. Initially $id[i] = i$.

- `find`$(p) = id[p]$ – $O(1)$.
- `union`(p, q): if $id[p] \neq id[q]$, scan the *entire* array and relabel every i with $id[i] = id[p]$ to $id[q]$ – $O(n)$.

Proposition 7.1. *With Quick-Find, a sequence of $n-1$ unions (enough to merge n singletons into one component) can cost $\Theta(n^2)$ total time in the worst case, since each union can touch $\Theta(n)$ array entries.*

Problem 18 implements `QuickFind` from scratch, including instrumentation counters (`find_calls`, `union_relabels`) used to measure this cost empirically.

7.2 Quick-Union

Quick-Union instead stores a *parent* array $parent[0..n-1]$, where each element initially points to itself ($parent[i] = i$), forming n trees (each a single node, its own **root**).

- `find`(p): follow *parent* pointers from p until reaching a node that is its own parent (the **root**) – O(tree height).
- `union`(p, q): set $parent[$`find`$(p)] \leftarrow$ `find`(q) (attach one root under the other) – O(tree height).

Proposition 7.2. *With naive Quick-Union (always attaching the root of p's tree under the root of q's tree, regardless of size), a carefully chosen sequence of unions can build a tree of height $\Theta(n)$ (e.g. a path: union(1,0), union(2,1), union(3,2), ...), making `find` cost $\Theta(n)$ in the worst case.*

7.3 Union by rank

Union by rank (or union by size) fixes the linear-chain problem: maintain a **rank** (an upper bound on tree height) for each root, and when unioning, attach the *shorter* tree under the root of the *taller* tree (breaking ties arbitrarily, and incrementing the surviving root's rank only when the two ranks were equal).

Lemma 7.3. *With union by rank (no path compression), every tree has height $O(\log n)$, where n is the total number of elements.*

Proof sketch. By induction, a tree with rank r has at least 2^r nodes. A rank-0 tree is a single node ($2^0 = 1$, true). When two trees of ranks $r_1 \leq r_2$ are merged, the result has rank $\max(r_1, r_2)$ if $r_1 < r_2$, or $r_1 + 1$ if $r_1 = r_2$; in either case the new tree's node count is at least the sum of the two subtrees' node counts $\geq 2^{r_1} + 2^{r_2} \geq 2 \cdot 2^{\min(r_1,r_2)} = 2^{\min(r_1,r_2)+1}$, which is $\geq 2^{\text{new rank}}$ in both cases. Since a tree on n elements has at most n nodes, its rank r satisfies $2^r \leq n$, i.e. $r \leq \log_2 n$. The height of a tree is at most its rank, so `find` costs $O(\log n)$. □

7.4 Path compression

Path compression is an additional optimization applied during `find`: after locating the root r of x, make every node visited on the way from x to r point *directly* to r. This flattens the tree for future operations, at the cost of a second pass during `find`.

Theorem 7.4 (Union-Find with union by rank and path compression)**.** *A sequence of m `find`/`union` operations on n elements, using union by rank* and *path compression, takes $O(m \log^* n)$ total time, and in fact $O(m\alpha(n))$ where α is the inverse Ackermann function – for all practical n, $\alpha(n) \leq 4$, so each operation is effectively $O(1)$* ***amortized****.*

We will not prove the full inverse-Ackermann bound (it is a famously intricate analysis), but the key intuition is: path compression means that once a node's path to the root is "flattened", subsequent finds through that node are $O(1)$; combined with union by rank's $O(\log n)$ height bound, the *total* flattening work across m operations is bounded by a function that grows astronomically slowly – $\log^* n$ (the number of times you must apply log to n before reaching ≤ 1) is at most 5 for any n that could ever be represented in a real computer.

The `UnionFind` class in `starter_code.py` implements exactly this combination (union by rank + path compression) over arbitrary hashable elements, and is used as a building block by later weeks (e.g. Kruskal's MST algorithm).

7.5 Applications

- **Dynamic connectivity**: answer "are u and v connected?" queries while edges are only ever added (never removed) – exactly `find` and `union`.
- **Kruskal's Minimum Spanning Tree algorithm** (a later week): process edges in increasing order of weight, adding an edge iff its endpoints are in different components (checked with `find`), then merging them (`union`).
- **Detecting cycles while building a graph incrementally**: an edge (u, v) creates a cycle iff `find`$(u) =$ `find`(v) *before* the edge is added.
- **Image segmentation / connected-component labeling**: pixels (or grid cells) are unioned with their neighbors of similar value; the resulting components correspond to connected regions.

8 Summary and Looking Ahead

This week established:

- Two graph representations (adjacency list, adjacency matrix) and their $\Theta(n+m)$ vs. $\Theta(n^2)$ space trade-off.
- BFS: shortest paths/distances/layers in unweighted graphs, and connected components, in $O(n+m)$.
- DFS (recursive and iterative): connected components, cycle detection, and discovery/finish times, also $O(n+m)$.
- Bipartiteness via 2-coloring, and the odd-cycle characterization (Theorem 5.2).
- Directed graphs: 3-color DFS for cycle detection, and two $O(n+m)$ algorithms for topological sort (DFS-based and Kahn's).
- Union-Find: from $\Theta(n)$-per-union Quick-Find, to $O(\log n)$ union by rank, to near-$O(1)$ amortized with path compression (Theorem 7.4).

Next week (Kleinberg & Tardos, Chapter 4) we use BFS as a stepping stone toward **greedy algorithms** – including weighted shortest paths (Dijkstra's algorithm) and minimum spanning trees, where Union-Find reappears as the core data structure behind Kruskal's algorithm.